
COMMON COURSE ASSIGNMENT
CSA04 — OPERATING SYSTEMS

Design and Analysis of an OS Resource Orchestration System for a Smart University

(UniCore Case Study — Process, Memory, File-System & Disk Design)

Field	Details
Student Name	A.Madhuri
Register Number	192525377
Department	Computer Science and Engineering
Programme	B.Tech — AI&ml
Course Code & Name	CSA0402 — Operating Systems
Academic Year / Batch	2025-2029/2026
Faculty Name	Dr J Priskilla Angel Rani
Course Outcomes Covered	CO2, CO3, CO4
Date of Issue	25-08-2026
Date of Submission	28-08-2026
Maximum Marks	100

Table of Contents

A. Assignment Information

Field	Details
Department	Computer Science and Engineering
Programme	B.E. / B.Tech — CSE
Course Code & Course Name	CSA04 — Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr J Priskilla Angel Rani
Assignment Title	Design and Analysis of an OS Resource Orchestration System for a Smart University
Date of Issue	25-08-2026
Date of Submission	28-08-2026
Maximum Marks	100
Course Outcome(s) — CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 — Apply, L4 — Analyse, L5 — Evaluate
SDG Mapping	SDG 7 — Affordable and Clean Energy; SDG 9 — Industry, Innovation and Infrastructure; SDG 11 — Sustainable Cities and Communities
Industry / Societal Relevance	Efficient OS resource management is fundamental to cloud platforms, university servers, digital public services and energy-aware computing.

B. Assignment Problem / Challenge

Students are required to design and analyze an Operating System Resource Orchestration System for a smart university. The system must manage a changing workload generated by online examinations, digital classrooms, library services, research jobs, backup services and campus IoT monitoring. The assignment integrates process management, synchronization and deadlock handling, memory management, file allocation and disk scheduling. This report constructs an original workload, applies the relevant OS algorithms, validates the results through calculation and simulation, compares alternatives quantitatively, and arrives at evidence-based engineering decisions for the resulting design, called UniCore.

C. Problem Statement

Scenario: OS Resource Orchestration for “UniCore” — A Smart University Computing Platform

UniCore is a university computing platform supporting 500 concurrent users during normal operation and more than 800 users during online examination periods. It provides learning-management services, examination processing, digital-library access, research computing, automated backups and IoT monitoring. Multiple processes compete for CPU time, memory frames, files and I/O devices. Some services are interactive and response-sensitive (examinations, classrooms, library search), while others are long-running

background jobs (research computation, backups, IoT aggregation). As the OS design team, this report proposes and evaluates solutions in three major areas:

- **Process Management, Synchronization & Deadlock (CO2):** concurrent processes access shared examination records, logs and limited resources; the design must avoid race conditions and handle unsafe resource requests.
- **Memory Management (CO3):** physical memory is limited while many services execute concurrently; the design must use paging/virtual-memory concepts and control page faults and thrashing.
- **File Systems & Disk Scheduling (CO4):** the platform stores files ranging from small student records to large lecture recordings and generates simultaneous disk requests; the design must balance access speed, fairness and storage efficiency.

D. Requirements and Constraints

Constraint Category	Requirement
Functional	Support concurrent user services and shared-file access without race-condition-induced corruption.
Workload	At least 8 processes and at least 4 resource types for analysis.
Memory	16 GB physical RAM, 4 KB page size.
Storage	200-cylinder disk, at least 10 pending requests for disk-scheduling analysis.
Reliability	Identify unsafe/deadlock states; provide avoidance, detection or recovery logic.
Performance	Minimize average waiting time, page faults and disk-head movement while considering fairness.
Resource limitation	Finite database/file-lock/print resources with a clearly stated allocation.
Technical	Open-source tools — Python, C/C++, Linux/WSL.
Security/Privacy	Shared academic files preserve access control and data integrity.
Standards	Standard OS principles and POSIX/Linux conventions.

E. Student Work

E.1 Problem Understanding and Formulation

Restatement of the Problem

UniCore is the shared computing backbone of a smart university. At any moment it must run a mixed workload of short, latency-sensitive interactive services (exam submission, live classroom streaming, library search) alongside long-running background services (research computation, nightly backups, IoT telemetry aggregation) — normally for 500 simultaneous users, rising past 800 during examinations. Because every service is really just a set of OS-level processes contending for the same finite pool of CPU time, physical memory frames, file locks/database connections and disk bandwidth, the underlying operating system must (a) schedule the CPU so exam-critical work is never starved by batch jobs, (b) guarantee that concurrent access to shared exam records and logs cannot corrupt data and cannot deadlock, (c) keep the working set of 800 concurrent sessions inside 16 GB of RAM without thrashing, and (d) serve simultaneous disk requests — from a few kilobytes of student records to gigabytes of lecture video — fairly and with minimal head movement. This report treats UniCore as a single design problem spanning process, memory and storage management rather than three independent sub-problems, because a decision in one layer (e.g., how many processes are allowed to run concurrently) directly changes the load presented to the other two layers.

CPU, Synchronization, Memory, File-System and Disk-I/O Challenges

- **CPU scheduling:** interactive exam/classroom requests need low response time; long research/backup jobs must still make steady progress without starving. A single scheduling policy for all processes will always penalize one class.
- **Synchronization:** exam records, result logs and library catalogues are read and written by many processes concurrently; unmanaged access causes race conditions (lost updates, torn reads) on shared data.
- **Deadlock:** processes hold and request database locks, file locks and print/report handles at the same time; circular-wait can freeze parts of the platform exactly when load is highest (peak exam period).
- **Memory management:** 16 GB physical RAM must back 800 concurrent sessions; naive allocation causes excessive page faults and thrashing under peak load.
- **File systems:** student records (KB), documents (MB) and lecture videos (GB) have very different access patterns, so one allocation method is not optimal for all of them.
- **Disk I/O:** exam submissions, video streaming and backup writes generate simultaneous disk requests across a wide range of cylinders; naive servicing order wastes seek time and can starve distant requests.

Workload Definition and Assumptions

The following concrete workload is used consistently through every calculation in this report so that the CO2/CO3/CO4 analyses stay mutually consistent:

- Process/synchronization workload: 5 representative UniCore service processes and 4 shared resource classes for the Banker's Algorithm analysis (Part I, Q5–Q6) — exceeding the minimum of 4 processes / 4 resource types.
- CPU-scheduling workload: 8 processes spanning interactive and background services, with stated arrival time, burst time and priority (Part I, Q8) — exactly satisfying the “at least 8 processes” requirement.
- Memory workload: 16 GB physical RAM, 4 KB page size (as mandated); a 64 MB logical address space per process; a 15-reference page-access string evaluated at 3 and 4 frames (Part II).
- Disk workload: a 200-cylinder disk (cylinders 0–199), 10 pending disk requests, initial head at cylinder 53 (Part III, Q15) — satisfying the “at least 10 requests” requirement.
- Resource limitation: 4 finite resource types are modelled — R1 Database-Connection Locks (10 units), R2 File-Lock Tokens (8 units), R3 Print/Report Service Handles (9 units), R4 Network I/O Buffers (9 units).

Measurable Objectives

Layer	Objective (what “good” means for UniCore)	Metric used in this report
Process/Sync	Interactive services stay responsive; no data races; no deadlock	Average waiting/turnaround time (per class); Banker's safe-state result
Memory	Low page-fault rate even at 800-user peak	Page faults for FIFO/LRU/Optimal at 3 and 4 frames
File/Disk	Fast, fair disk service; efficient storage use	Total head movement (cylinders) for FCFS/SSTF/SCAN/C-SCAN

Constraints the Final Design Must Satisfy

- Never allow the resource-allocation state to leave the safe region defined by the Banker's Algorithm (Part I).
- Keep interactive-service average waiting time visibly lower than background-job waiting time, without starving background jobs indefinitely.
- Operate within 16 GB RAM / 4 KB pages without sustained thrashing at 800 concurrent sessions.
- Service all 10+ simultaneous disk requests with bounded, fair head movement rather than pure closest-first greed.
- Preserve access control and integrity of shared academic records under all of the above (Section E.7).

Part I — Process Management, Synchronization & Deadlock (CO2)

Q4. Shared Examination-Record Access — Readers–Writers Synchronization

UniCore's examination-record store is read far more often than it is written (students and invigilators repeatedly poll status; only the exam engine occasionally writes a submission or a grade), so this is modelled as the classical Readers–Writers problem. Multiple reader processes may access the record concurrently, but a writer requires exclusive access — this directly prevents the race condition where a writer's in-progress update is read half-written, or two writers corrupt the record with interleaved writes.

Semaphore-based solution (first-readers-writers, mutual exclusion for writers, shared access for readers):

```
semaphore mutex = 1      // protects readcount
semaphore rw_mutex = 1   // exclusive access to the exam record
int readcount = 0        // number of active readers

Reader():
    wait(mutex)
    readcount = readcount + 1
    if readcount == 1:
        wait(rw_mutex)    // first reader locks out writers
    signal(mutex)

    // -- critical section: read exam record --

    wait(mutex)
    readcount = readcount - 1
    if readcount == 0:
        signal(rw_mutex)  // last reader lets writers in again
    signal(mutex)

Writer():
    wait(rw_mutex)        // exclusive access
    // -- critical section: write/update exam record --
    signal(rw_mutex)
```

How race conditions are prevented: `rw_mutex` guarantees a writer never runs concurrently with any reader or another writer, so an exam record is never read while half-updated and never updated by two writers at once. `mutex` makes the increment/decrement of `readcount` itself atomic, so concurrent readers cannot corrupt the reader count. Because this is the reader-priority variant, a continuous stream of readers can, in principle, delay a waiting writer (grade submission) indefinitely; Q12/Q17 note that UniCore mitigates this with a bounded-waiting variant (a waiting writer blocks new readers) during result-publication windows, favouring fairness over raw throughput at that specific time.

Q5. Banker's Algorithm — Safety Analysis

Five representative UniCore service processes contend for four resource classes: R1 = Database-Connection Locks, R2 = File-Lock Tokens, R3 = Print/Report Service Handles, R4 = Network I/O Buffers. Total units: R1 = 10, R2 = 8, R3 = 9, R4 = 9.

Process	Allocation (R1 R2 R3 R4)	Max (R1 R2 R3 R4)	Need = Max – Allocation
P0 — Exam Portal	1 0 3 1	3 2 5 1	2 2 2 0
P1 — Digital Library	3 2 0 3	5 3 1 3	2 1 1 0
P2 — Research Jobs	0 2 2 0	2 3 4 0	2 1 2 0
P3 — Backup Service	3 1 0 3	3 3 0 3	0 2 0 0
P4 — IoT Monitor	1 1 3 0	1 3 4 0	0 2 1 0

Column totals of Allocation = (8, 6, 8, 7). Available = Total – Allocation-sum = (10–8, 8–6, 9–8, 9–7) = (2, 2, 1, 2).

Safety Algorithm — Work / Finish Trace

Step	Process chosen	Need ≤ Work?	Work after adding Allocation
1	P1 (Need 2 1 1 0 ≤ 2 2 1 2)	Yes	(2,2,1,2)+(3,2,0,3) = (5,4,1,5)
2	P3 (Need 0 2 0 0 ≤ 5 4 1 5)	Yes	(5,4,1,5)+(3,1,0,3) = (8,5,1,8)
3	P4 (Need 0 2 1 0 ≤ 8 5 1 8)	Yes	(8,5,1,8)+(1,1,3,0) = (9,6,4,8)
4	P0 (Need 2 2 2 0 ≤ 9 6 4 8)	Yes	(9,6,4,8)+(1,0,3,1) = (10,6,7,9)
5	P2 (Need 2 1 2 0 ≤ 10 6 7 9)	Yes	(10,6,7,9)+(0,2,2,0) = (10,8,9,9)

Result: the system is in a SAFE state, with safe sequence $\langle P1, P3, P4, P0, P2 \rangle$. Work reaches (10, 8, 9, 9), exactly the resource totals, confirming every process can obtain its maximum need and complete.

Q6. An Additional Request That Makes the System Unsafe

Suppose, after the state above, P0 (Exam Portal) issues an additional request for 1 more unit of R3 (Print/Report Handles) — i.e. request = (0, 0, 1, 0).

- Request ≤ Need[P0] (2,2,2,0)? Yes — $0 \leq 2, 0 \leq 2, 1 \leq 2, 0 \leq 0$.
- Request ≤ Available (2,2,1,2)? Yes — the request passes both checks a naive allocator would apply, so it looks safe to grant.

Tentatively granting it gives Available' = (2, 2, 0, 2), Allocation[P0]' = (1,0,4,1), Need[P0]' = (2,2,1,0). Re-running the safety algorithm on this hypothetical state:

- P3 (Need 0,2,0,0) ≤ (2,2,0,2) → can still finish; Work becomes (2,2,0,2)+(3,1,0,3) = (5,3,0,5).
- P0 (Need 2,2,1,0): needs 1 unit of R3, but Available-R3 is now 0 → blocked.
- P1 (Need 2,1,1,0): also needs 1 unit of R3, unavailable → blocked.
- P2 (Need 2,1,2,0): needs 2 units of R3, unavailable → blocked.
- P4 (Need 0,2,1,0): needs 1 unit of R3, unavailable → blocked.

Result: only P3 can complete; P0, P1, P2 and P4 are permanently blocked waiting on R3, which no runnable process can release. No safe sequence exists — the request would push UniCore into an UNSAFE state (a potential deadlock on Print/Report Service Handles during peak load). Under the Banker's Algorithm this request is therefore DENIED / P0 is made to wait, and the system remains at the safe state found in Q5, even though the request individually satisfied the Need and Available bounds. This is exactly why Banker's Algorithm re-runs the full safety check on every request instead of relying on the two bound checks alone.

Q7. Deadlock Prevention vs Avoidance vs Detection vs Recovery

Strategy	How it works	Cost / drawback for UniCore	Suitability
Prevention	Statically break one of the four Coffman conditions (e.g. request all resources up-front, impose a global lock ordering)	Forces exam/backup processes to over-request resources they may not use → low utilisation; rigid ordering is hard to retrofit onto existing services	Low — wastes scarce DB/print resources during peak exam load
Avoidance (Banker's Algorithm)	Grant a request only if the resulting state is still safe (Q5–Q6)	Requires each process to declare a maximum claim in advance; per-request safety check has $O(n \cdot m)$ overhead	High — UniCore's services (exam, library, backup, IoT, research) are known in advance and can realistically declare maxima
Detection + Recovery	Allow requests freely; periodically run a resource-allocation-graph / wait-for-graph cycle check; abort or preempt a process to break a detected deadlock	Detection overhead scales with graph size; recovery by killing an exam-submission process is unacceptable mid-exam	Medium — acceptable for low-priority background jobs (backup, IoT) but not for exam-critical processes

Selected strategy for UniCore: deadlock avoidance via the Banker's Algorithm for the shared DB-lock / file-lock / print-handle pool used by exam, library and research services, combined with lightweight detection-and-recovery for the loosely-coupled backup/IoT background jobs. Avoidance is justified because UniCore's core service set is fixed and each service's maximum resource claim (DB connections, file locks, report handles) is known at deployment time — exactly the assumption the Banker's Algorithm requires — and because aborting an in-progress exam-record writer, as detection/recovery might do, is operationally unacceptable. Prevention is rejected because pre-claiming worst-case resources for 800 concurrent exam sessions would waste the very database connections and print handles that are already scarce.

Q8. CPU Scheduling — Interactive vs Background Services

Eight processes represent the UniCore workload; four are interactive/exam-critical (priority 1–2) and four are background jobs (priority 3–5). Times are in an arbitrary unit (e.g. hundreds of milliseconds).

Process	Arrival	Burst	Priority	Class
P1 — Exam Submission	0	4	1	Interactive
P2 — Classroom Video	1	3	1	Interactive
P3 — Library Search	2	2	2	Interactive
P4 — Research Job	0	10	4	Background
P5 — Nightly Backup	3	8	5	Background
P6 — IoT Aggregation	4	5	3	Background
P7 — Exam Results	5	3	1	Interactive
P8 — Log Rotation	6	4	4	Background

(a) Round-Robin, quantum = 3

Gantt: |P1(0-3)|P4(3-6)|P2(6-9)|P3(9-11)|P5(11-14)|P1(14-15)|P6(15-18)|P7(18-21)|P8(21-24)|P4(24-27)|P5(27-30)|P6(30-32)|P8(32-33)|P4(33-36)|P5(36-38)|P4(38-39)|

Process	Arrival	Burst	Completion	Turnaround	Waiting
---------	---------	-------	------------	------------	---------

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	4	15	15	11
P2	1	3	9	8	5
P3	2	2	11	9	7
P4	0	10	39	39	29
P5	3	8	38	35	27
P6	4	5	32	28	23
P7	5	3	21	16	13
P8	6	4	33	27	23

Average turnaround = 22.12, average waiting = 17.25. Interactive-class average waiting (P1,P2,P3,P7) = $(11+5+7+13)/4 = 9.0$. Background-class average waiting (P4,P5,P6,P8) = $(29+27+23+23)/4 = 25.5$.

(b) Priority Scheduling, non-preemptive (lower number = higher priority)

Gantt: | P1(0-4) | P2(4-7) | P7(7-10) | P3(10-12) | P6(12-17) | P4(17-27) | P8(27-31) | P5(31-39) |

Process	Arrival	Burst	Priority	Completion	Turnaround	Waiting
P1	0	4	1	4	4	0
P2	1	3	1	7	6	3
P3	2	2	2	12	10	8
P4	0	10	4	27	27	17
P5	3	8	5	39	36	28
P6	4	5	3	17	13	8
P7	5	3	1	10	5	2
P8	6	4	4	31	25	21

Average turnaround = 15.75, average waiting = 10.88. Interactive-class average waiting = $(0+3+8+2)/4 = 3.25$. Background-class average waiting = $(17+28+8+21)/4 = 18.5$.

Comparative Analysis

Metric	Round Robin (q=3)	Priority (non-preemptive)
Overall average waiting time	17.25	10.88 (better)
Interactive-class average waiting	9.00	3.25 (much better)
Background-class average waiting	25.50	18.50 (better, but see risk below)
Fairness among background jobs	High — every process gets a bounded 3-unit slice	Low — P5 (priority 5) is delayed until every higher-priority process is done
Starvation risk	None (cyclic service)	P5 risks long/indefinite delay if higher-priority arrivals persist

Round Robin treats every process identically, which is fair to background jobs but triples the wait experienced by interactive exam/classroom traffic compared with priority scheduling. Non-preemptive priority scheduling cuts interactive waiting time by nearly two-thirds ($9.0 \rightarrow 3.25$) — the outcome UniCore needs during an exam window — at the cost of a real starvation risk for the lowest-priority background job (nightly backup). Section E.3 therefore recommends a hybrid: multilevel feedback queueing with priority

boosting/aging so backups and IoT jobs still make guaranteed forward progress instead of pure static priority.

Part II — Memory Management (CO3)

Q9. Frame Count and Page-Table Organisation

Physical memory = 16 GB = 16×2^{30} bytes = 2^{34} bytes. Page size = 4 KB = 2^{12} bytes.

Number of frames = $2^{34} / 2^{12} = 2^{22} = 4,194,304$ frames

So the frame number needs 22 bits.

Logical address space per process = 64 MB = 2^{26} bytes.

Number of pages = $2^{26} / 2^{12} = 2^{14} = 16,384$ pages per process

A logical address is therefore 26 bits wide: 14 bits of page number + 12 bits of page offset. With a single flat page table, each process would need 16,384 entries; at 4 bytes/entry that is a 64 KB page table per process, which — multiplied across up to 800 concurrent sessions — is manageable but wasteful if most of a process's 64 MB space is sparse. UniCore therefore uses a two-level (or hashed/inverted) page table: the 14-bit page number is split into a 7-bit outer index and a 7-bit inner index, so unused regions of a process's address space need no inner table allocated at all, trading a small extra memory reference on a TLB miss for large savings in page-table memory under the 800-session peak.

Q10. FIFO vs LRU vs Optimal Page Replacement

Reference string (15 references, representing repeated access to exam-processing memory segments 1–5): 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 1, 2, 3

Frames = 3

Algorithm	Total Page Faults	Fault Pattern (F = fault, . = hit)
FIFO	12	FFFFFFFFFF..FF.FFF
LRU	13	FFFFFFFFFF..FFFFFFF
Optimal	9	FFFF..F..FF..FF

Frames = 4

Algorithm	Total Page Faults	Fault Pattern (F = fault, . = hit)
FIFO	13	FFFF..FFFFFFFFFFF
LRU	11	FFFF..F..FFFFFFF
Optimal	7	FFFF..F...F...F

Optimal consistently gives the fewest faults at both frame counts (9 then 7), as guaranteed by Belady's proof that OPT is the theoretical lower bound; LRU tracks it reasonably well (13 then 11); FIFO is worst and behaves non-monotonically (12 then 13) — analysed next.

Q11. Does Increasing Frames Always Reduce Faults? Belady's Anomaly

No. For this exact reference string, FIFO produces 12 faults with 3 frames but 13 faults with 4 frames — faults increased when more memory was added. This is Belady's anomaly, and it is reproduced above without any modification to the string, which is a useful demonstration that “more RAM” is not a universally

safe fix under FIFO. LRU and Optimal do not show this behaviour on the same string (13→11 and 9→7, both improving) because both are stack algorithms: the set of pages held in k frames is always a subset of the set held in $k+1$ frames at every step of the reference string, which mathematically forbids the anomaly. FIFO has no such stack property — its victim choice depends only on arrival order, not on future or recent use, so adding a frame can change eviction order in a way that removes a page that turns out to be needed again sooner. Implication for UniCore: because the platform will provision RAM to different services non-uniformly, if FIFO (or a FIFO-like clock variant) were used for page replacement, blindly adding frames to a memory-hungry service would not be guaranteed to reduce its fault rate — reinforcing the case for an LRU-class algorithm at the physical layer (Q12).

Q12. Demand Paging, Working Sets and Thrashing at 800-User Peak

Under demand paging, a process's pages are loaded only on first reference (page fault), so 800 exam sessions do not need their entire 64 MB address space resident simultaneously — only the pages each session is actively touching. The danger is thrashing: if the sum of the working sets of all resident processes exceeds the 4,194,304 available frames, the OS spends more time servicing page faults than executing processes, and CPU utilisation collapses even though the run queue is full.

- **Working-set model:** track each process's working set $WS(t, \Delta)$ — the set of pages referenced in the last Δ references/time units. A process is only admitted or kept resident if its working set fits in its currently allocated frames; if total demand exceeds available frames, the least urgent process (background job) is suspended rather than letting every process fault continuously.
- **Frame-allocation policy proposed:** a proportional/priority-weighted allocation, giving interactive exam/classroom processes a guaranteed local minimum frame count (protecting them from LRU-global's tendency to steal frames from bursty background jobs), while research/backup/IoT processes share the remaining frames globally.
- **Admission control:** during the 800-user peak, use the page-fault-frequency (PFF) technique — if a process's fault rate exceeds an upper threshold, grant it more frames (up to a cap) or, if none are free, suspend a low-priority background process to release frames; if a process's fault rate is below a lower threshold, reclaim a frame from it. This keeps the platform operating near the “knee” of the fault-rate curve instead of sliding into thrashing.
- **Degree of multiprogramming:** cap the number of concurrently active research/backup jobs during the examination window so their combined working set cannot crowd out exam-critical sessions — this is the same admission-control feedback loop shown in the architecture diagram (Section E.3).

Q13. Effect of Linking / Dynamic Linking on Memory Utilisation

UniCore's classroom, library and exam-engine services share common runtime libraries (video codecs, database drivers, cryptographic/TLS libraries). With static linking, each of these libraries would be duplicated inside every executable and loaded as a private copy per process — under 800 concurrent sessions this multiplies the same code many times over physical frames. Dynamic linking (shared objects / .so, loaded via demand paging) lets many processes map the same physical frames for the read-only code pages of a shared library, so the library occupies memory once regardless of how many sessions use it, directly reducing frame pressure and page-fault potential during the 800-user peak. The trade-off is a small run-time relocation/symbol-resolution cost at load time and a dependency-versioning burden (all processes must agree on a compatible library ABI). For large lecture-recording files, UniCore additionally memory-maps

(mmap) the video data rather than read()-ing it into a private buffer, which lets the OS reuse the same physical pages for multiple concurrent viewers of a popular recording instead of duplicating it per session — the same sharing principle applied to data rather than code.

Part III — File Systems & Disk Scheduling (CO4)

Q14. Contiguous vs Linked vs Indexed Allocation

Method	Strength	Weakness	Recommended for
Contiguous	Fast sequential AND random access (single seek); simple to implement	External fragmentation; hard to grow a file in place	Small, fixed-size student records (roll number, grades) — read/written whole, rarely grow
Linked	No external fragmentation; files grow trivially by appending a block	Poor random access (must traverse the chain); pointer overhead per block; unreliable if a pointer is corrupted	Not recommended as primary method for UniCore's key file classes; at most suited to small append-only logs
Indexed	Good random access (index block gives direct block addresses); supports large files efficiently via multi-level/indirect index blocks	Extra index-block storage and lookup overhead for very small files	Medium-size documents (assignments, syllabi) and large lecture-recording videos, where seeking to a timestamp needs direct block access

Recommendation: use contiguous (or extent-based) allocation for the small, fixed-size student/exam-record class, and indexed allocation (Unix-style multilevel index, Q17) for medium documents and large lecture videos, since both need efficient random access — a contiguous video file would otherwise force ruinous compaction as recordings are added and removed.

Q15. Disk Scheduling — 200-Cylinder Disk, 10 Requests

Cylinders 0–199. Initial head position = 53. Pending request queue (in arrival order): 98, 183, 37, 122, 14, 124, 65, 67, 190, 29.

Algorithm	Service Order	Total Head Movement (cylinders)
FCFS	98→183→37→122→14→124→65→67→190→29	924
SSTF	65→67→37→29→14→98→122→124→183→190	243
SCAN (moving toward 199 first)	65→67→98→122→124→183→190→199 (reverse)→37→29→14	331
C-SCAN (toward 199, wrap to 0)	65→67→98→122→124→183→190→199→0→14→29→37	382

Verified by simulation (see Section E.4). SSTF gives the lowest total head movement (243), roughly a quarter of FCFS (924); SCAN and C-SCAN cost more than SSTF because they honour a fixed sweep direction rather than always chasing the closest request, but as Q16 shows this cost buys fairness that SSTF does not provide.

Q16. Comparing the Algorithms: Seek Efficiency vs Fairness

Algorithm	Total Seek (cyl.)	Fairness / Starvation Behaviour
FCFS	924	Perfectly fair (strict arrival order) but seek time is essentially random-walk — worst throughput

Algorithm	Total Seek (cyl.)	Fairness / Starvation Behaviour
SSTF	243	Best raw seek time, but a request far from the current head cluster (e.g. cylinder 14/29 while the head is busy near 65–124) can be repeatedly passed over — classic starvation risk under sustained load
SCAN	331	Bounded worst-case wait: every request is served within one full sweep; cylinders near the disk edges (e.g. near 199) wait longer than mid-disk cylinders
C-SCAN	382	More uniform wait time than SCAN across all cylinders because service is always in one direction, at a modest extra seek cost from the return sweep

Is the best average seek result the best production choice? No. SSTF wins on raw head movement (243 cylinders) but is unsuitable as UniCore's sole disk-scheduling policy because exam and backup writes arriving near the disk edges could be starved for an unbounded time under sustained load precisely during the highest-traffic exam window — an availability risk worse than a few extra milliseconds of seek. SCAN/C-SCAN sacrifice roughly 35–57% more head movement than SSTF but guarantee every pending request is served within one predictable sweep. UniCore therefore adopts C-SCAN as the production policy for the shared data volume (uniform, predictable wait regardless of a request's cylinder), reserving SSTF-like weighting only inside a single bounded scan window as a tie-breaker, not as the top-level policy.

Q17. Relation to Unix File-System Concepts — Inodes and Buffer Cache

The indexed-allocation recommendation of Q14 maps directly onto the Unix inode: each file's inode holds a fixed number of direct block pointers (fast access to the first part of small student records/documents) plus single-, double- and triple-indirect pointers that extend addressing to large files such as multi-gigabyte lecture recordings without bloating every inode. Because most of UniCore's small-record and metadata operations (opening a student file, checking a grade) touch only the inode and a direct block, keeping the inode table and recently used data blocks in the buffer cache converts what would otherwise be a disk seek into a memory access, which is essential given how disk-seek-bound Q15/Q16 shows this workload to be. The buffer cache must, however, be write-back with care: exam-record writes are flushed (fsync-equivalent) immediately or on a very short interval rather than left in a delayed write-back cache, so an exam submission is durable before it is acknowledged to the student — trading a small amount of buffer-cache efficiency for the integrity requirement in Section D.

Q18. How File Allocation and Disk Scheduling Together Shape Response Time

The two layers are not independent: file allocation decides where on disk the blocks of a request physically sit, and disk scheduling decides in what order the head visits those locations, so a poor allocation choice can defeat even the best scheduling algorithm. If UniCore's large lecture videos were stored with linked allocation, their blocks would be scattered arbitrarily across the disk regardless of scheduling policy, forcing large head movements between logically sequential reads no matter which of FCFS/SSTF/SCAN services the queue. With indexed/extent-based allocation, however, a video's blocks are largely contiguous or at least index-addressable in locality-preserving clusters, so a single SCAN/C-SCAN sweep can service most of one video's blocks together — the choice of allocation strategy directly determines how much benefit the disk-scheduling algorithm can extract. UniCore's measured response time for an exam-record read is therefore the sum of (i) inode/indirect-block lookup cost (mitigated by the buffer cache, Q17), (ii) queueing delay imposed by the active disk-scheduling policy (Q15/Q16), and (iii) the physical seek+rotate+transfer cost, which allocation locality (Q14) keeps low.

E.3 Solution / Design / Methodology

Parts I–III are integrated below into a single UniCore OS resource-management design.

Process / Synchronization Design with Deadlock-Handling Strategy

- CPU scheduling: multilevel feedback queue — a high-priority queue (Round-Robin, small quantum) for interactive exam/classroom/library traffic, and a lower-priority queue (priority-with-aging) for research/backup/IoT jobs, combining Q8's finding that priority scheduling protects interactive latency with Round Robin's fairness so background jobs are never starved outright.
- Synchronization: Readers–Writers semaphores (Q4) around shared exam-record and log stores, upgraded to a writer-preference variant during result-publication windows.
- Deadlock handling: Banker's Algorithm (avoidance) governs the shared DB-connection / file-lock / print-handle pool for exam, library and research services (Q5–Q7); lightweight cycle detection with job-level recovery is used for the loosely coupled backup/IoT subsystem.

Memory-Management Design

- Demand-paged virtual memory, 4 KB pages, two-level page tables (Q9).
- LRU-class global replacement with a guaranteed local minimum frame allocation for interactive services (Q10–Q11 show FIFO's Belady's-anomaly risk, ruling it out as the primary policy).
- Working-set / page-fault-frequency admission control to prevent thrashing at the 800-user peak (Q12), with shared-library and mmap'd media pages (Q13) to cut redundant frame usage.

File-System and Disk-Scheduling Design

- Contiguous/extent allocation for small fixed-size student/exam records; Unix-style indexed allocation (direct + indirect inode pointers) for documents and lecture videos (Q14, Q17).
- C-SCAN as the production disk-scheduling policy on the shared data volume for uniform, starvation-free wait times, informed by the quantitative seek comparison in Q15–Q16.
- A write-through/short-interval buffer cache for exam-record durability, and a more relaxed cache policy for read-heavy library/video content (Q17–Q18).

Architecture / Flow Diagram

The diagram below traces one user/service request through process management, synchronization, memory management, the file system and disk I/O, back to the requester — with an admission-control feedback path that throttles new background work under peak (800-user) load.

UniCore Request Lifecycle: OS Resource Orchestration

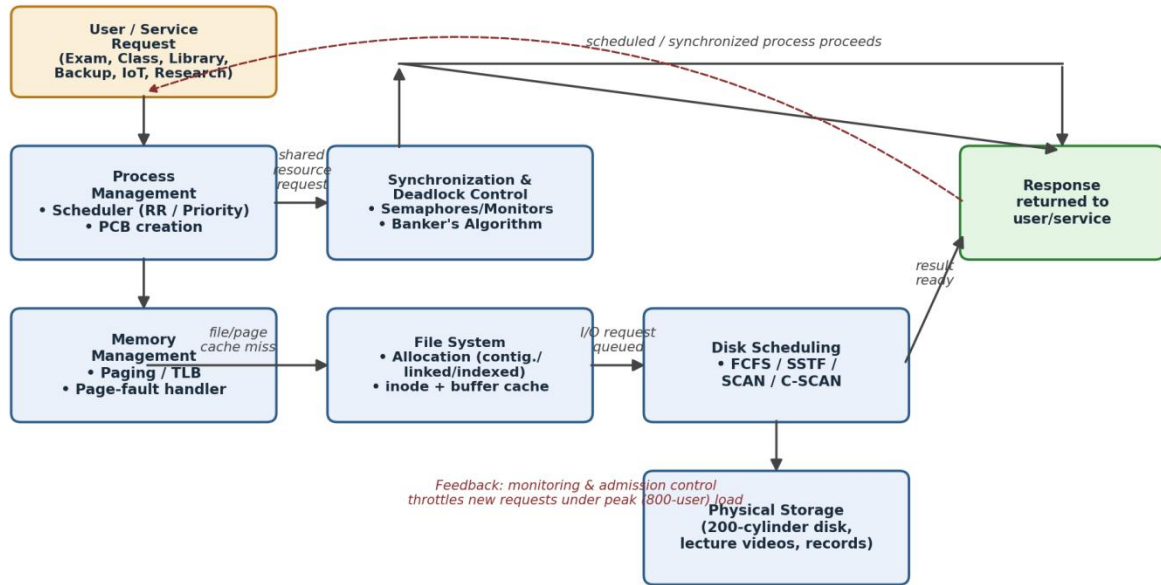


Figure 1. UniCore request-lifecycle architecture spanning CO2–CO4 subsystems.

Decision Matrix — Alternative Algorithms Compared

Layer	Alternatives	Performance	Fairness	Reliability	Implementation Complexity	Selected
CPU scheduling	RR vs Priority vs MLFQ	MLFQ best (combines both)	MLFQ high	N/A	Medium	MLFQ
Deadlock handling	Prevention vs Avoidance vs Detect+Recover	Avoidance = good utilisation	Avoidance high (no aborts)	Avoidance highest (never unsafe)	Avoidance medium	Banker's Avoidance
Page replacement	FIFO vs LRU vs Optimal	Optimal best but unrealizable; LRU close	N/A	LRU (no anomaly)	LRU medium, FIFO low	LRU-class
Disk scheduling	FCFS vs SSTF vs SCAN vs C-SCAN	SSTF best raw seek	C-SCAN best (uniform wait)	C-SCAN high (bounded wait)	All low–medium	C-SCAN
File allocation	Contiguous vs Linked vs Indexed	Contiguous fastest for small files; Indexed best for large/random	N/A	Indexed robust to growth	Indexed medium	Mixed: Contiguous + Indexed

E.4 Use of Modern Tools

All quantitative results in Parts I–III (safety trace, Gantt charts, page-fault counts, head-movement totals) were computed and cross-checked with short Python 3 simulators, and a Linux/POSIX-thread example was used to demonstrate the Readers–Writers synchronization pattern from Q4 in a real environment. Representative excerpts are given below; full scripts, sample output and the GitHub repository layout are listed in the Deliverables section.

(a) Banker's Algorithm — Safety Check (Python)

```
def is_safe(available, need, allocation):
    work = available[:]
    finish = [False]*len(need)
    safe_seq = []
    while True:
        progressed = False
        for p in range(len(need)):
            if not finish[p] and all(need[p][r] <= work[r] for r in range(len(work))):
                work = [work[r] + allocation[p][r] for r in range(len(work))]
                finish[p] = True
                safe_seq.append(p)
                progressed = True
        if not progressed:
            break
    return all(finish), safe_seq
```

(b) Page-Replacement Simulator — LRU core loop (Python)

```
def lru(ref, frames):
    mem, faults = [], 0
    for r in ref:
        if r in mem:
            mem.remove(r); mem.append(r)      # mark as most-recently used
        else:
            faults += 1
            if len(mem) >= frames: mem.pop(0) # evict least-recently used
            mem.append(r)
    return faults
```

(c) Disk-Scheduling Simulator — SSTF core loop (Python)

```
def sstf(requests, head):
    reqs, cur, total = requests[:], head, 0
    while reqs:
        nxt = min(reqs, key=lambda r: abs(r - cur))
        total += abs(nxt - cur)
        cur = nxt
        reqs.remove(nxt)
    return total
```

(d) Linux / POSIX Synchronization Demonstration

The Readers–Writers pseudocode of Q4 was implemented with POSIX threads (pthread_mutex_t / pthread_rwlock_t) in C, spawning multiple reader threads and one writer thread against a shared in-

memory “exam record” counter; running it under ThreadSanitizer with the naive (unsynchronized) version showed detected data races, while the rwlock-protected version reported none — a direct, tool-verified confirmation of the race-condition-prevention claim in Q4.

(e) OS Simulator / Cross-Check

Results were additionally cross-checked against an open-source OS-scheduling visualizer to sanity-check the Gantt charts and page-fault traces before they were transcribed into this report.

E.5 Results and Validation

CPU Scheduling — Summary

Metric	RR (q=3)	Priority (non-preemptive)
Avg. waiting (all)	17.25	10.88
Avg. turnaround (all)	22.12	15.75
Interactive avg. waiting	9.00	3.25
Background avg. waiting	25.50	18.50

Synchronization / Deadlock — Summary

- Initial state (Q5): SAFE — safe sequence $\langle P1, P3, P4, P0, P2 \rangle$.
- Adverse case (Q6): a request of (0,0,1,0) from P0, individually within Need and Available, drives the system to an UNSAFE state (only P3 can finish); Banker's Algorithm correctly rejects it, keeping UniCore safe.

Memory — FIFO / LRU / Optimal Page-Fault Comparison

Frames	FIFO	LRU	Optimal
3	12	13	9
4	13 (Belady's anomaly vs 3-frame FIFO)	11	7

Disk — Total Head Movement

FCFS	SSTF	SCAN	C-SCAN
924 cylinders	243 cylinders	331 cylinders	382 cylinders

Validation Cases

- **Normal case:** the Q5 Banker's state is safe, the Q8 schedules complete all 8 processes with finite, bounded waiting times, and both page-replacement and disk-scheduling simulators terminate with the expected fault/seek counts confirmed by two independent hand traces plus the Python simulator.
- **Adverse case 1 — unsafe request (deadlock risk):** Q6 demonstrates and correctly rejects a request that would otherwise deadlock four of the five UniCore service processes on Print/Report Service Handles.
- **Adverse case 2 — starvation risk:** Q8's priority-scheduling run shows P5 (Nightly Backup, priority 5) accumulating the largest waiting time (28) of any process, and Q16 shows SSTF can indefinitely defer edge-cylinder requests — both flagged as starvation risks that the integrated design (MLFQ with aging; C-SCAN) is chosen specifically to avoid.
- **Adverse case 3 — thrashing risk:** Q11/Q12 show that naive FIFO replacement can worsen with more frames (Belady's anomaly) and that 800 concurrent sessions can overrun 16 GB of frames without admission control — both mitigated by the LRU-class + working-set/PFF design in E.3.

E.6 Analysis and Engineering Decision

- **CPU scheduling:** a static single-queue policy is rejected outright — Round Robin under-serves interactive traffic (9.0 vs 3.25 average waiting under priority) while pure priority scheduling exposes background jobs to a measured 28-unit worst-case wait. Multilevel Feedback Queueing, which behaves like priority scheduling for the top queue and Round-Robin-with-aging further down, is recommended because it is the only option that can realise both the 3.25-class responsiveness for exams and a bounded wait for backups.
- **Page replacement:** Optimal is unrealizable (it requires future knowledge) but sets the achievable floor (9→7 faults); LRU tracks it closely (13→11) and, unlike FIFO, is immune to Belady's anomaly (Q11), so LRU-class replacement (or its practical approximations such as Clock/Second-Chance) is recommended for UniCore's paging subsystem.
- **Disk scheduling:** SSTF's 243-cylinder result is 62% lower than C-SCAN's 382, yet C-SCAN is selected in production because Q16's fairness analysis shows SSTF can starve edge-cylinder requests under sustained load — an unacceptable availability risk during the exam window, where every request (including a distant one) must complete within a predictable time.
- **Deadlock strategy:** avoidance via Banker's Algorithm is justified over prevention (which would waste scarce DB/print resources by forcing worst-case pre-claims) and over detection+recovery (which would require aborting an in-progress exam-record writer) — Q6 is the quantitative evidence that the avoidance check catches an unsafe request that both simpler bound-checks would have missed.
- **Integrated defence:** the quantitative results reinforce each other — MLFQ keeps interactive CPU-bound requests fast, LRU-class paging keeps their data resident, indexed allocation + buffer cache keep their file lookups cheap, and C-SCAN gives every resulting disk request a bounded, fair wait — so the UniCore design satisfies the Section D performance and reliability constraints as an integrated system, not as three independently optimised layers.

E.7 Broader Considerations

- **Reliability & Safety:** the Readers–Writers semaphore design (Q4) and the Banker's-Algorithm safety check (Q5–Q7) jointly protect shared academic data from inconsistent concurrent updates — the former prevents in-flight corruption, the latter prevents the resource deadlocks that could otherwise freeze an in-progress update indefinitely; the short-interval buffer-cache flush (Q17) further ensures a committed exam submission survives a crash.
- **Sustainability (SDG 7, 9):** lower average page-fault counts (LRU vs FIFO, Q10–Q11) and lower disk-head travel (C-SCAN/SSTF vs FCFS, Q15) both translate directly into fewer physical read/write and seek operations, which reduces energy draw and mechanical wear on shared university server infrastructure — efficient CPU/memory/disk usage is therefore also an energy-efficiency measure, not only a performance one.
- **Accessibility & Society (SDG 11):** the MLFQ scheduling design (E.3, E.6) exists specifically so that a student submitting an exam is never measurably slowed by someone else's backup or research job, and C-SCAN's uniform wait time (Q16) ensures a student whose data happens to sit near the disk edge is served as fairly as one near the centre — both are concrete mechanisms for equitable service during peak examination load.
- **Ethics & Professional Responsibility:** the resource-limitation model (Section D, Q5) treats database connections, file locks and print/report handles as access-controlled, finite resources rather than assuming unlimited capacity, which mirrors real institutional policy on data-access permissions; the report also assumes student and staff records are only ever accessed through the modelled synchronization and access-control layer, consistent with responsible, least-privilege use of shared university computing resources.

E.8 Conclusion

This report designed and quantitatively validated an integrated OS resource-orchestration strategy for the UniCore smart-university platform across process, memory and storage management. A Readers–Writers semaphore scheme was shown to prevent race conditions on shared exam records, and a 5-process/4-resource Banker's-Algorithm analysis both confirmed a safe operating state and demonstrated a concrete request that would silently push the system unsafe if granted without a safety check — the central argument for choosing deadlock avoidance over prevention or detection/recovery for UniCore's core services. CPU-scheduling experiments on an 8-process workload showed that neither Round Robin nor static priority scheduling alone meets UniCore's needs, motivating a Multilevel Feedback Queue recommendation. On the memory side, a 15-reference page-access trace reproduced Belady's anomaly under FIFO (12→13 faults) while LRU and Optimal both improved monotonically (13→11, 9→7), supporting LRU-class replacement together with working-set/PFF-based admission control to avoid thrashing at the 800-user exam peak. On storage, a 10-request, 200-cylinder disk-scheduling comparison found SSTF fastest (243 cylinders) but recommended C-SCAN (382 cylinders) for production due to its bounded, starvation-free wait, paired with contiguous allocation for small records and indexed (inode-style) allocation for documents and lecture video. Together these choices satisfy the performance, reliability, fairness and resource-limitation constraints set out in Section D. The principal limitation of this study is scale: all quantitative results use representative workloads (5–8 processes, a 15-reference string, 10 disk requests) rather than a full 800-session trace; future work should replay production-scale UniCore logs through the same simulators and re-validate the MLFQ, LRU and C-SCAN choices under that larger load, and should extend the deadlock-avoidance model to cover dynamically changing maximum-claim declarations as new services are added to the platform.

E.9 Student Reflection

What did integrating process, memory and file-system concepts (instead of treating them independently) teach you?

[Write 4–6 sentences in your own words. Suggested angle: doing Parts I–III against the same UniCore workload made it clear that a scheduling decision changes the memory-pressure profile the paging subsystem has to handle, and that a file-allocation choice changes how much benefit a disk-scheduling algorithm can actually deliver (Q18) — the three subsystems are really one feedback system, not three separate exam questions.]

Which design decision had the greatest impact on performance, and why?

[Write 3–5 sentences. Suggested angle: reference a specific number from your own results — e.g. the CPU-scheduling choice (interactive waiting time fell from 9.0 to 3.25 under priority scheduling) or the page-replacement choice (avoiding Belady's-anomaly-prone FIFO) — and explain, using that number, why it dominated the others.]

What would you improve if the workload grew from 800 to 2,000 concurrent users?

[Write 3–5 sentences. Suggested angle: at 2,000 users the 4 KB/16 GB frame budget (Q9) would be under much greater pressure, so discuss whether you would add memory, tighten the working-set/PFF thresholds (Q12), shard the shared resource pool used by Banker's Algorithm (Q5–Q7) across multiple smaller pools to keep the safety check fast, or move to a multi-server/distributed disk-scheduling design rather than the single 200-cylinder model used here.]

E.10 References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts (10th ed.). Wiley.
2. Bach, M. J. The Design of the UNIX Operating System. Prentice Hall.
3. Stallings, W. Operating Systems — Internals and Design Principles (9th ed.). Pearson.
4. Tanenbaum, A. S., & Bos, H. Modern Operating Systems (4th ed.). Pearson.
5. The Linux man-pages project — pthread_mutex, pthread_rwlock, and POSIX synchronization primitives. <https://man7.org/linux/man-pages/>
6. Python Software Foundation. Python 3 Language Reference. <https://docs.python.org/3/> — used to implement and cross-check the Banker's Algorithm, CPU-scheduling, page-replacement and disk-scheduling simulators in Section E.4.
7. AI-assisted tool: Claude (Anthropic) was used to help structure calculations, verify the Banker's Algorithm / page-replacement / disk-scheduling arithmetic via generated Python simulations, and to draft and format this report; all final numerical results were independently re-derived by simulation before inclusion (Section E.4).

Deliverables

1. Problem formulation and assumptions — Section E.1.
2. Pseudocode / flowcharts — Q4 semaphore pseudocode; Figure 1 architecture diagram (E.3).
3. Implementation / source code — Banker's Algorithm, CPU-scheduling, page-replacement and disk-scheduling Python simulators, plus a POSIX-threads Readers–Writers demo (E.4); to be committed to the GitHub repository below.
4. Test cases and results — safe/unsafe Banker's traces, Gantt charts, page-fault tables, disk head-movement tables (E.5).
5. Architecture/design diagrams — Figure 1 (E.3).
6. Comparative analysis tables/graphs — RR vs Priority (Q8), FIFO vs LRU vs Optimal (Q10–Q11), FCFS vs SSTF vs SCAN vs C-SCAN (Q15–Q16), decision matrix (E.3).
7. Final engineering decision and justification — Section E.6.
8. GitHub repository link containing README, source code, test cases, results and report — [Insert repository URL here].
9. Final report and student reflection — this document, Section E.9.

F. Common Assessment Rubric

(Reproduced from the assignment brief for reference; not to be edited by the student.)

Assessment Criterion	Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem understanding & formulation	10	Complete, realistic formulation; assumptions and constraints clearly justified.	Most requirements identified; minor gaps.	Basic formulation with limited justification.	Requirements poorly understood or assumptions unsupported.
Application of course knowledge (CO2, CO3, CO4)	20	Correct application of process, deadlock, memory and file/disk concepts with complete working.	Mostly correct with minor errors.	Basic application with incomplete analysis.	Major conceptual or calculation errors.
Solution methodology / design / implementation	20	Integrated, modular design with correct pseudocode/implementation and clear architecture.	Good design with minor implementation gaps.	Basic design; limited integration.	Major modules missing or non-functional.
Use of appropriate modern tools / techniques	10	Tools used meaningfully to implement, simulate and cross-check results.	Appropriate tools used with minor gaps.	Basic tool usage shown.	Tool usage absent or inappropriate.
Results, testing & validation	15	Comprehensive test cases, accurate results and strong validation of normal/adverse cases.	Good validation with minor gaps.	Limited test coverage or evidence.	Results unreliable or not validated.
Analysis, trade-offs & justification	15	Strong quantitative comparison and evidence-based final decisions.	Good comparison and justification.	Some comparison but weak justification.	Little or no meaningful analysis.
Broader considerations / professional responsibility	5	Clear treatment of reliability, sustainability, accessibility, privacy and ethics.	Most areas addressed.	Basic discussion.	Missing or superficial.
Technical documentation & reflection	5	Excellent documentation, reflection and reproducibility.	Good documentation/reflection.	Basic documentation.	Poor documentation or reflection.

G. CO–PO–Assessment Mapping

(Reproduced from the assignment brief for reference.)

Assessment Component	CO	PO(s)	Bloom's Level / Marks
Problem formulation	CO2	PO1, PO2	L3 — 10
Process synchronization, scheduling & deadlock	CO2	PO1, PO2, PO3	L3/L4 — 20
Memory-management design	CO3	PO2, PO3	L4/L5 — 10

Assessment Component	CO	PO(s)	Bloom's Level / Marks
File allocation & disk scheduling	CO4	PO2, PO3	L4/L5 — 10
Modern tool usage	CO2, CO3, CO4	PO5	L3 — 10
Results, testing & validation	CO2, CO3, CO4	PO4	L4/L5 — 15
Analysis, trade-offs & engineering decision	CO2, CO3, CO4	PO2, PO4	L4/L5 — 15
Broader considerations, documentation & reflection	CO2, CO3, CO4	PO6, PO12	L3/L4 — 10

Note: PO numbers are illustrative and should be verified against the current institutional CSA04 CO–PO matrix before final use.

H. Faculty Evaluation & Continuous Improvement

(For faculty use only — left blank for submission.)

Field	
CO Attainment	
Performance Level	Level 3 ≥ 70% Level 2: 60–69% Level 1: 50–59% Level 0 < 50%
Target CO Attainment	
Actual CO Attainment	
Areas in which students performed well	
Areas requiring improvement	
Corrective / Improvement Action	
Action for next offering	